ARK Platform — ATLAS 360 Living PDD

Current Codebase Baseline · Reverse-Engineered Product, Project & Program Document

> ***Living document status:*** *Current repository baseline*
> ***Version:*** *1.0.0 · 03 September 2026*
> ***Production ID:*** `JNGL-ARK-PDD-LIVE-2026-012`
> ***Honesty Gate:*** *Tier 1 — codebase-verified, not independently production-certified*
> ***GRO mode:*** *HUMAN_IN_LOOP*
> ***Document owner:*** *ATLAS 360 PLAN format · ATANDA Publications Forum / Koncentric Labs Inc.*
> ***Human principal:*** *Oluseye Shay Amusa*

This document is a present-tense map of the ARK Platform codebase. It is not a proposal for an unbuilt product, a certification claim, a legal opinion, or a financial forecast. It records what the repository currently contains, what local verification has established, and what remains inferred or unverified.

PART 0 · Title, Contents & Provenance

Contents

1. Single-Page Cheat Sheet
2. Executive Summary
3. SPC Roster and System Inventory
4. Platform Specifics
5. Atomic Prompt Worksheet
6. DJ Implementation and Maintenance Plan
7. Summary of Sources
8. Definition of Terms
9. IP Pre-Registration Screening
10. Living Delta and Review Rules

Authorship and provenance block

FIELD	VALUE
Document title	ARK Platform — ATLAS 360 Living PDD · Current Codebase Baseline
Production ID	`JNGL-ARK-PDD-LIVE-2026-012`
Snapshot date	03 September 2026
Source	Current workspace repository, current `replit.md`, current schema/flags/routes, local test and typecheck results
Reference method	ATLAS 360 PLAN v3.0.0, including its 10-part structure and Honesty Gate
Document mode	Living PDD / reverse translation of the codebase
Certification	No JCSE score assigned; not HIVE/FORGE certified by this document
Review cadence	After each merged feature, schema migration, route-surface change, or stage/flag change
Human review	Required for inferred business intent, production posture, compliance conclusions, and IP findings

Scope boundary

The source repository demonstrates implementation structure. It does not, by itself, demonstrate production traffic, customer outcomes, production database state, live third-party register searches, legal compliance, or deployment health over time. Those claims are intentionally excluded unless separately evidenced.

PART 1 · Single-Page Cheat Sheet

Audience: All stakeholders · **Scan target:** 60 seconds

FIELD	CURRENT STATE
Product	ARK Platform — Advanced Resume & Karriere Synthesized Intelligence
Current stage	Stage 1 · MVP, with all current feature flags defaulting on and reversible by server environment overlay
What it does	Builds a scored career-intelligence identity from assessment and resume evidence, then provides skill practice, verification, career pathways, marketplace, workforce, reporting, and guidance surfaces
Canonical identity	**ARK 0–600 = JST 0–300 + CCMI 0–300**
JST	Jobs–Skills–Talent index: `(Jobs × .30 + Skills × .40 + Talent × .30) × 3`
CCMI	Context Craft Mastery Index: weighted P1–P7 pillar score × 3
Main user loop	Resume/assessment → ARK identity → pathways and practice → CCGE/verification activity → capped score growth and recorded outcomes
Current guide	MERLIN, available persistently in the authenticated app shell and at `/merlin`; provides current position, destination preference, and next action
Current report	ARK Report plus ARK Guide; report/export surfaces are distinct from MERLIN guidance
Client surface	35 page modules and 43 declared client routes, including the authenticated platform, public demo/report/share/confirmation paths, and feature-gated surfaces
Server surface	165 route declarations in `server/routes.ts`; exact externally reachable endpoint count should be regenerated if route composition changes
Persistence	PostgreSQL through Drizzle ORM; server startup applies idempotent SQL migrations and tracks them in `schema_migrations`
Current migrations	25 SQL migration files through `0023_preferred_ai_model.sql`; the duplicate `0020_` prefix is retained as repository history and must not be reused
Feature flags	24 static feature keys; all default `true`; server overlays can disable a flag without a rebuild
AI	Multi-provider layer for Anthropic, OpenAI, and Gemini integrations with model policy, fallback chains, usage logging, token and cost guardrails
Auth	Express sessions backed by PostgreSQL; password hashing at the storage boundary; server-derived actor identity and `requireSelf` authorization

Verification	Local typecheck and full suite pass at this snapshot: 293 tests passing
Deployment claim	Development workflow is configured and running; no production deployment certification is claimed here

Atomic solution summary

ARK is a full-stack career-intelligence platform. It combines a market-facing **JST** signal with a Context Craft **CCMI** signal, composes them into a bounded ARK identity, and surrounds that identity with activities intended to make growth observable rather than merely aspirational. The current codebase also contains enterprise workforce tools, HR connector adapters, training-provider matching, marketplace economics, shareable reporting, ARK Resume eligibility, verification documents, and the MERLIN completion-assurance layer.

Non-claims carried forward

- ARK does not claim blockchain or NFT badge verification.
- Primitive verification rows are not automatically described as public book badges.
- Training-provider catalog certifications are recommendations, not credentials issued by ARK.
- Development tests do not prove production scale, customer conversion, or regulatory compliance.
- A codebase snapshot does not establish that a feature is enabled in every deployed environment.

PART 2 · Executive Summary

2.1 Problem and product intent

Knowledge workers need a way to understand how their existing work maps to an AI-changing labor market, identify the skills that matter next, and see evidence of progress. Institutions need bounded workforce views rather than unrestricted personal-data access. ARK addresses these needs with a scored identity and a set of guarded pathways:

1. **Measure:** resume and assessment inputs become JST, vulnerability, archetype, transferability, and Context Craft signals.
2. **Orient:** the Intelligence Hub, pathways, ARK Report, ARK Guide, and MERLIN explain the current position and likely next action.
3. **Practice:** CCGE skill games, Context Craft levels, book-journey quests, and verification quests provide structured work.
4. **Prove:** recorded CCGE tier outcomes, earned book-journey badges, verification results, and ARK Resume eligibility expose only supported evidence.
5. **Connect:** SPHINX marketplace, GUIN+ profiles, matchmaking, training providers, cohorts, and workforce tools extend the identity into relevant contexts.

2.2 Current requirements

PRIORITY	REQUIREMENT	PRESENT IMPLEMENTATION EVIDENCE
MUST	Keep one canonical ARK identity and invariant	`shared/schema.ts`, `server/scoringEngine.ts`, `server/arkRecalc.ts`
MUST	Derive actor identity from the authenticated session	`server/auth.ts`, `requireAuth`, `requireSelf`, route ownership checks
MUST	Keep score-growth caps server-side and transactional	`server/arkRecalc.ts`, flywheel source caps, transactional storage
MUST	Make feature gating fail closed	`shared/featureFlags.ts`, `server/featureFlags.ts`, client route gating
MUST	Avoid unsupported credential and legal claims	ARK Guide, MERLIN, report copy, platform documentation boundaries

MUST	Keep persistence changes migration-backed	`server/migrate.ts` , `migrations/*.sql` , `schema_migrations`
SHOULD	Push current identity changes to the client	Orchestrator SSE and `useArkStream`
SHOULD	Keep AI usage bounded and attributable	provider chains, usage ledger, model-aware cache keys, token/cost gates
SHOULD	Give users direct next steps rather than only scores	`/pathways` , `/play` , `/book` , `/training` , `/merlin` , `/ark-resume`
NICE	Support institution-scale operations	workforce aggregation, department filters, HR connector adapters, nudge flow

2.3 Capability matrix

CAPABILITY	USER VALUE	CURRENT ROUTE / ENGINE	STATUS
ARK identity and scoring	One bounded career-intelligence identity	`/dashboard` , `scoringEngine.ts` , `arkRecalc.ts`	Implemented
Resume analysis	Career signals, vulnerability, archetype, transferability	`/upload` , `resumeAnalyzer.ts`	Implemented
Career pathways	Pivots, upskilling timeline, skill gaps	`/pathways`	Implemented
Context Craft practice	Repeated skill practice with recorded outcomes	`/play` , `ccge.ts`	Implemented
Book Companion	Guided reading journey and canonical badges	`/book` , `bookCompanion.ts`	Implemented, flag-controlled
Primitive verification	Evidence-gated card verification	`/api/verification/*` , verification UI	Implemented, flag-controlled
ARK Resume	ATS-oriented resume artifact and confirmation layer	`/ark-resume`	Implemented, eligibility-gated
ARK Report / Guide	Shareable report plus personalized explanation	`/report` , `/shared-report`	Implemented, plan/report-gated
MERLIN	Completion assurance with destination	`/merlin` , app-shell copilot	Implemented in current snapshot

	preference		
Training providers	JST-matched catalog and affiliate path	`/training`	Implemented, provider access-gated
SPHINX marketplace	Prompt/IP listing, credits, purchase economics	`/marketplace`, `sphinx.ts`	Implemented
GUIN+ and matchmaking	Public identity and verified-evidence matching	`/u/:username`, `/matchmaking`	Implemented, flag-controlled
Cohorts / workforce	Instructor and institution workflows	`/school`, `/enterprise`, `/workforce`	Implemented, role/flag-controlled
Billing	Subscription-shaped entitlement flow	`/subscription`, `/checkout/:id`	Implemented as a stub, not a live processor

2.4 Risk register

RISK	SEVERITY	CURRENT CONTROL	OPEN EVIDENCE GAP
Score farming or cap bypass	High	Single-writer recalc, source caps, row/transaction controls, regression tests	Production adversarial monitoring
Cross-user data access	High	Session identity, `requireSelf`, scoped queries, institution boundary checks	Independent penetration test
AI cost overrun	High	Token and cost gates, plan policy, usage ledger, resilient provider chain	Production budget telemetry
Unsupported credential language	High	MERLIN/report copy distinguishes earned, recorded, and recommended outcomes	Content review after every new outcome
Feature flag drift	Medium	Shared defaults, server overlay, route gates, fail-closed middleware	Cross-environment flag inventory
PII or resume exposure	High	Authenticated APIs, DTO boundaries, security headers, GDPR export/delete	External privacy review
Connector failure or stale imports	Medium	Adapter contract, connection testing, import batch records	Provider-specific live audit

Billing stub mistaken for production billing	Medium	Documentation and route-level stub behavior	Live payment integration and reconciliation
PDD drift	Medium	This document’s review rules and source ledger	Automated code-to-PDD diffing

2.5 Business and financial boundary

The codebase contains real billing-shaped routes, credits, marketplace splits, sponsored-provider ordering, and affiliate-click recording. This PDD does not assign revenue, ROI, savings, valuation, or conversion forecasts because no verified operating dataset was supplied. Any future financial section must distinguish code paths from measured business results.

PART 3 · SPC Roster and System Inventory

3.1 PDD method roster

CONTRIBUTOR / METHOD	ROLE IN THIS DOCUMENT	STATUS
ATLAS 360 PLAN	Ten-part assembly, atomic decomposition, current-state translation	Method applied
OSIRIS-style living review	Drift bands and re-baselining discipline	Design rule applied; no automated monitor claimed
CODE DJ	Reverse translation from code structure into platform and atomic rows	Human/agent execution role
KLARITY pass	Plain-language requirements and non-technical summaries	Applied editorial rule
CARTER grounding	Stable terms and duplicate-term avoidance	Applied to core glossary; no external terminology registry instantiated
ALBERT layer	Candidate IP screening section	Included as an unrun, non-advisory boundary

No contributor roster above is a claim that those SPCs are live software dependencies in the ARK application.

3.2 Codebase inventory

INVENTORY ITEM	OBSERVED VALUE AT SNAPSHOT
Client page modules	35
Declared client route lines	43
Server TypeScript files	89
Shared TypeScript files	7

SQL migrations	25
Static feature keys	24
Dedicated server test files	23
Full test assertions passing	293
Application port	5000
Client/server topology	Vite client served through Express in one development process

Counts are repository inventory figures, not availability or performance claims. The route line count includes conditional and overlapping declarations; route composition should be re-counted after routing changes.

3.3 Repository ownership map

DOMAIN	PRIMARY CODE LOCATIONS	CONTRACT
Runtime and security	`server/index.ts`, `server/auth.ts`, `server/static.ts`	Express bootstrap, sessions, security headers, rate limits
Persistence	`server/storage.ts`, `server/db.ts`, `shared/schema.ts`, `migrations/`	Drizzle/PostgreSQL and migration-backed schema
Scoring	`server/scoringEngine.ts`, `server/arkRecalc.ts`, `server/lhcs.ts`	JST, CCMI, ARK invariant, caps, readiness
Events	`server/orchestrator.ts`, `client/src/lib/useArkStream.ts`	Typed event bus, SSE updates, client merge
AI	`server/ai/*`, `server/profileExtract.ts`, `server/resumeAnalyzer.ts`	Provider resolution, budgets, caching, usage
Flywheel	`server/ccge.ts`, `server/cardVerification.ts`, `server/bookCompanion.ts`	Practice, verification, badges, score-impact events
Marketplace	`server/sphinx.ts`, `server/sphinxSynergy.ts`, `server/zpos.ts`	Listings, HIVE precheck, credits, purchases, synthesis
Workforce and connectors	`server/workforceConnectionTest.ts`, `server/hrScheduler.ts`, `server/*workforce*`, `server/*hr*`	Institution scope, adapters, imports, intelligence

Client shell	`client/src/App.tsx`, `client/src/components/layout/AppLayout.tsx`	Routing, navigation, persistent MERLIN surface
Client intelligence	`client/src/pages/dashboard.tsx`, `client/src/pages/pathways.tsx`, `client/src/pages/merlin.tsx`	User-facing identity, destination, and next action
Reports	`client/src/pages/report.tsx`, `client/src/pages/adviser-report.tsx`, `client/src/lib/arkReportExport.ts`	ARK Report, ARK Guide, exports and shares

PART 4 · Platform Specifics

4.1 User journeys

Individual journey

1. User registers or signs in.
2. User uploads a supported resume or completes the assessment flow.
3. The server persists assessment inputs and derives the ARK identity.
4. Dashboard and pathways expose scores, vulnerability, transferability, readiness, and next areas.
5. User practices in CCGE, completes evidence-gated verification, follows book-journey quests, or explores training.
6. Server-side flywheel rules cap score growth and record supported outcomes.
7. ARK Resume, ARK Report, ARK Guide, and MERLIN expose appropriate artifacts and next actions.

Instructor and institution journey

Authenticated, role-checked institution users can manage cohorts, assignments, grades, workforce views, connector configurations, imports, department drill-downs, name search, and upskill nudges. Institution access is scoped; a user’s individual data is not treated as organization-wide data without the relevant role and institution boundary.

Creator / marketplace journey

Creators prepare listings, pass HIVE checks and certification gates, publish supported prompt artifacts, receive purchases, and participate in credit/royalty records. Listing bodies and public profiles are subject to ownership, visibility, and plan controls.

4.2 Identity and score model

```
JST = (Jobs × 0.30 + Skills × 0.40 + Talent × 0.30) × 3
CCMI = weighted(P1…P7) × 3
ARK = JST + CCMI
```

```
0 ≤ JST ≤ 300
0 ≤ CCMI ≤ 300
0 ≤ ARK ≤ 600
```

ARK recalculation is centralized in `server/arkRecalc.ts`. Activity-based score changes are capped by source and time window. Manual recomputation and backfill paths are not allowed to award positive score. The current codebase treats the invariant as a server concern, not a client-calculated trust signal.

4.3 Outcome and credential semantics

The platform has multiple evidence types and they must not be collapsed into one label:

- **Assessment:** a measured input and derived result.
- **CCGE outcome:** a recorded skill-game session result and, where applicable, a Context Craft tier outcome.
- **Book badge:** a canonical book-journey badge earned by its defined event.
- **Verification result:** an evidence-gated primitive verification record with its own score/tier rules.
- **Training recommendation:** a catalog option matched to a user's JST signals.
- **ARK Resume eligibility:** a product gate based on plan and verified evidence; not a professional license.

MERLIN and the ARK Guide must preserve these distinctions.

4.4 MERLIN completion-assurance layer

MERLIN is a current product surface, not an AI claim. It reads existing authenticated API responses through a React Query-backed client hook and builds a pure guidance model. It provides:

- **Where you are:** assessment presence, readiness when available, book-badge progress, and current certification state.
- **Where you could go:** local preference among fastest certification, complete badges, strengthen AI resilience, or career mobility.
- **What to do next:** a real route based on the current data and selected preference.

MERLIN's current preference is browser-local for the MVP. It is not represented as a new server field. Missing source responses degrade independently where possible; a completely unavailable data set produces an explicit error state rather than synthetic progress.

4.5 Security and governance

- Session identity is server-derived; mutation handlers must not trust body-supplied actor IDs.
- `requireSelf` protects user-scoped path parameters.
- Role and institution middleware protect instructor/admin/workforce paths.
- Feature flags return 404 for disabled server surfaces so gated features are indistinguishable from unimplemented routes.
- Security headers, request limits, session settings, and password handling are centralized.
- User export and cascade deletion are implemented as privacy operations; this document does not claim regulatory certification.

4.6 AI system

The provider layer supports Anthropic, OpenAI, and Gemini integrations. Model resolution considers task defaults, user preference, plan policy, provider availability, and resilience fallback. AI calls are budget-gated before provider invocation, logged with the model actually used, and cached using the resolved primary model rather than a hardcoded model literal. AI output is not used as a substitute for authorization or score invariants.

4.7 Feature activation

All 24 static keys in `shared/featureFlags.ts` currently default to `true`. This is an MVP activation posture, not proof that all commercial or production triggers have occurred. The server supports environment overlays; the client's static route registration follows the source defaults. Any new gated surface must be checked both at the parent route and at reserved child paths.

PART 5 · Atomic Prompt Worksheet

Each row is intentionally one operation with an observable input and output. The rows describe current code, not a backlog. A row marked **inferred** describes behavior assembled from source structure where a dedicated executable contract was not found.

Phase A · Foundation and authorization

ID	ATOMIC TASK	INPUT	OUTPUT	VALIDATION	OWNER
FND-001	Start the Express/Vite application	Environment and port	Running app on port 5000	Workflow reaches port	`server/index.ts`
FND-002	Apply PostgreSQL migrations at startup	Migration directory	Updated schema and migration ledger	Startup logs show migration state	`server/migrate.ts`
FND-003	Create a session-backed authenticated request	Credentials	Session cookie and user DTO	Auth tests and `/api/auth/me`	`server/auth.ts`
FND-004	Enforce self-scope on a user route	Session user and path ID	Allow or 403	Authorization tests	`server/auth.ts`
FND-005	Apply security headers and request limits	HTTP request	Bounded, protected response	Server configuration and route tests	`server/index.ts`

Phase B · Identity and scoring

ID	ATOMIC TASK	INPUT	OUTPUT	VALIDATION	OWNER
IDN-001	Compute JST from Jobs, Skills, and Talent	Three bounded sub-scores	JST 0–300	Scoring unit tests	`server/scoringEngine.ts`

IDN-002	Compute CCMI from seven pillar scores	P1–P7 values	CCMI 0–300	Scoring unit tests	`server/scoringEngine.ts`
IDN-003	Compose an ARK identity	JST and CCMI	ARK 0–600	Invariant tests	`server/scoringEngine.ts`
IDN-004	Recalculate and persist identity atomically	Event and user state	Updated user/pillar/history state	Recalc tests and transaction path	`server/arkRecalc.ts`
IDN-005	Publish an identity update event	Recalc event	SSE event and client merge	Orchestrator / stream path	`server/orchestrator.ts`
IDN-006	Compute LHCS readiness status	CPR, MPS, LCIS signals	Composite and band	Threshold tests	`server/lhcs.ts`

Phase C · Assessment and movement

ID	ATOMIC TASK	INPUT	OUTPUT	VALIDATION	OWNER
MOV-001	Parse an uploaded resume	Supported file upload	Plaintext resume	Upload validation	`server/resumeAnalyze`
MOV-002	Derive resume career signals	Plaintext	JST inputs, vulnerability, archetype	Analyzer tests	`server/resumeAnalyze`
MOV-003	Produce transferability directions	Assessment signals	12-vector map and pivots	Pathways render	`client/src/pages/pat`
MOV-004	Record an assessment completion	Assessment result	Persisted assessment and flywheel event	Route/integration tests	`server/routes.ts`
MOV-005	Explain an assessment in the ARK Guide	Assessment and identity data	Personalized narrative sections	Guide narrative tests	`client/src/pages/adv report.tsx`

Phase D · Practice, verification, and book journey

ID	ATOMIC TASK	INPUT	OUTPUT	VALIDATION	OWNER
FLW-001	Deal a CCGE hand	Scenario and card pool	Session with playable cards	CCGE route tests	`server/ccge.ts`
FLW-002	Finalize a CCGE session	Played cards and session	JCSE/KCSE breakdown and outcome	Transactional finish path	`server/storage.ts`
FLW-003	Cap a practice score delta	Event source and window	Applied delta ≤ cap	Revenue/flywheel tests	`server/arkRecalc.ts`
FLW-004	Create an evidence-gated verification quest	Verified matched card	Quest specification	Verification route and evidence rules	`server/cardVerification.`
FLW-005	Persist a verification result	Prompt evidence and score	Verification status/tier	Verification tests	`server/cardVerification.`
FLW-006	Resolve book-journey progress	User and canonical journey	Nodes, earned state, and badge metadata	Journey route	`server/bookCompanion.ts`
FLW-007	Award a canonical book badge	Supported platform event	Earned badge row	Journey event handlers	`server/bookCompanion.ts`

Phase E · Marketplace and growth

ID	ATOMIC TASK	INPUT	OUTPUT	VALIDATION
GRW-001	Run an HIVE listing precheck	Listing draft	HIVE/KCSE result and reasons	Marketplace tests

GRW-002	Publish an eligible listing	Approved draft and creator identity	Public listing	Certification and ownership gates
GRW-003	Execute a marketplace purchase	Buyer, listing, credits	Purchase and split ledger	Transaction tests
GRW-004	Match providers to a JST assessment	Assessment and provider catalog	Ranked providers with reasons	Match tests
GRW-005	Record a safe affiliate click	Provider/course and outbound URL	Validated redirect target and click row	HTTP(S)-only validation
GRW-006	Assemble a shareable ARK Report	Identity and assessment	Renderable report and export	Report surface
GRW-007	Build MERLIN guidance	Existing API results and local preference	Position, destination, milestones, next route	`server/__tests__/merlinGuidance.test.`

Phase F · Enterprise, privacy, and edge governance

ID	ATOMIC TASK	INPUT	OUTPUT	VALIDATION	OWNER
GOV-001	Scope workforce intelligence to an institution	Admin session and institution	Aggregated workforce view	Workforce authorization tests	`server/routes.ts`
GOV-002	Test an HR connector without importing data	Adapter config	Connection result	Connector test suite	`server/workforce`
GOV-003	Import roster records through an adapter	Adapter and import request	Batch and staff rows	Connector/import tests	`server/routes.ts`

GOV-004	Nudge an eligible staff member	Scoped staff row	Notification/email attempt and cooldown	Nudge route tests	`server/routes.ts`
GOV-005	Export user-owned data	Authenticated session	JSON export	Privacy route tests	`server/storage.t`
GOV-006	Delete user-owned data after confirmation	Authenticated session and `DELETE` confirmation	Cascade deletion	Privacy deletion tests	`server/storage.t`
GOV-007	Apply a feature flag to a surface	Flag key and request	Route or 404	Flag tests and reserved-path checks	`server/featureFl`

Traceability rule

Every future atomic task must retain a parent feature and every atomic prompt must retain a parent task. If a proposed change cannot identify its parent capability, it belongs first in a requirements diagnosis rather than directly in implementation.

PART 6 · DJ Implementation and Maintenance Plan

6.1 Current tool and runtime map

CONCERN	CURRENT SELECTION	PRESENT USE
Runtime	Node.js / TypeScript	Express server and Vite client
Frontend	React, Vite, TailwindCSS, wouter	Page surfaces and app shell
Backend	Express 5	API, auth, Vite integration, SSE
Data	PostgreSQL, Drizzle ORM	Durable user, assessment, event, marketplace, cohort, and connector state
AI	Replit-managed Anthropic/OpenAI/Gemini integrations	Scoring support, narratives, scenarios, extraction, resilient fallback
Quality	TypeScript compiler and Node test runner via `tsx`	Current typecheck and 293-test suite
PDD rendering	Markdown to PDF with Puppeteer/Chromium	Existing `scripts/renderPdd.ts` workflow
Hosting model	Replit workflow and autoscale deployment configuration	Development workflow verified; production status not claimed

6.2 Maintenance cadence

TRIGGER	REQUIRED PDD ACTION
New page or route	Update Parts 1, 3, 4, and the relevant worksheet rows; re-count route inventory
New API or mutation	Update security, endpoint ownership, worksheet, and risk register
Schema migration	Update inventory, persistence map, source ledger, and migration note
Feature flag change	Update activation table and parent/child route behavior
New score or credential	Add glossary entry and document the evidence boundary before UI copy

AI provider/model change	Update AI section, budget boundary, and cache-key rule
New external integration	Add source and data-flow disclosure; do not imply availability until tested
Production deployment	Add deployment evidence separately; do not convert local verification into production certification

6.3 Stage-gate view

GATE	CURRENT DISPOSITION
Codebase can start in the configured development workflow	Verified at snapshot
TypeScript compilation	Pass
Automated test suite	293 passing
Database schema startup migration	Startup logs report current migration state
Feature surfaces	All static defaults on; server overlays remain possible
Production deployment	Not independently verified by this PDD
Live performance / scale	Not measured by this PDD
JCSE/FORGE certification of this document	Not assigned

6.4 Known follow-on work

The following are intentionally not represented as current capabilities:

- Durable, cross-device MERLIN destination preferences.
- Automated code-to-PDD drift scoring.
- A live ALBERT/open-register IP search against this PDD’s candidate terms.
- Production payment-provider processing and reconciliation.
- Production-scale event bus or connector observability.

These are future work items, not hidden implementation assumptions.

PART 7 · Summary of Sources

7.1 Internal sources

SOURCE	USE	CONFIDENCE
`replit.md`	Architecture, score glossary, routes, flags, security and platform conventions	VERIFIED-INTERNAL
`shared/schema.ts`	Canonical schema, score constants, certification levels, plan data	VERIFIED-INTERNAL
`shared/featureFlags.ts`	Current static flag keys and defaults	VERIFIED-INTERNAL
`server/scoringEngine.ts`	JST/CCMI/ARK formulas	VERIFIED-INTERNAL
`server/arkRecalc.ts`	Single-writer and score-cap behavior	VERIFIED-INTERNAL
`server/auth.ts`	Session and authorization behavior	VERIFIED-INTERNAL
`server/routes.ts`	HTTP route declarations and endpoint behavior	VERIFIED-INTERNAL
`server/storage.ts`	Persistence and transaction boundaries	VERIFIED-INTERNAL
`server/ai/*`	Provider policy, budgets, caching, and usage	VERIFIED-INTERNAL
`client/src/pages/*`	Current user-facing route surfaces	VERIFIED-INTERNAL
`client/src/components/layout/AppLayout.tsx`	Navigation and persistent MERLIN mount	VERIFIED-INTERNAL
`client/src/lib/merlin-guidance.ts`	MERLIN recommendation model	VERIFIED-INTERNAL
`migrations/*.sql`	Schema evolution history	VERIFIED-INTERNAL

`server/__tests__/*.test.ts`	Local behavior evidence; 293 assertions pass at snapshot	VERIFIED-INTERNAL

7.2 Reference source

SOURCE	USE
`attached_assets/ATLAS_360_PLAN_v3_0_0_ALBERT_Integration_1788458849105.pdf`	10-part PDD authorship/p requirement Gate, atomic chain, living- distinction, a screening bo

7.3 Not used as proof

Historical PDD exports, prior task descriptions, design intent, product names, and future roadmap statements are not proof that a current route or capability remains live. When they conflict with current code, current code wins and this document should record the discrepancy.

PART 8 · Definition of Terms

TERM	DEFINITION IN THIS PDD	TYPE
ARK	Bounded composite identity equal to JST plus CCMI, with a 0–600 range	Metric
JST	Jobs–Skills–Talent market-readiness index, 0–300	Metric
CCMI	Context Craft Mastery Index across the seven Context Craft pillars, 0–300	Metric
LHCS	Readiness composite derived from CPR, MPS, and LCIS signals	Metric
CCGE	Context Craft game / arena that produces practice-session outcomes	Product engine
JCSE	Quality score/tier vocabulary used for session or artifact outcomes; the exact rubric depends on context	Outcome metric
KCSE	In-game Knowledge, Clarity, Specificity, Efficiency rubric used to evaluate CCGE play	Outcome metric
HIVE	Listing/content quality gate used by marketplace publishing	Governance metric
MERLIN	Platform-wide Completion Assurance Guide that translates supported progress into a current position, chosen destination, and next action	Product surface
ARK Guide	User-facing companion narrative that explains an ARK Report in personalized language	Product surface
Book badge	A canonical badge awarded by a defined book-journey event; not interchangeable with every verification record	Credential-like outcome
Verification	Evidence-gated assessment of a supported primitive/card; not automatically a public badge	Evidence record
Flywheel	The set of platform activities and event handlers that can update identity or recorded progress	System concept
Single writer	The architectural rule that identity mutation is centralized through the recalc path	Governance rule
Feature flag	A server/client activation control; disabled server paths fail as 404	Configuration control
Explorer / PRO / school	Plan or entitlement vocabulary used by billing and feature access	Commercial state

Living PDD	A PDD refreshed from current code and evidence rather than treated as a one-time plan	Document mode
Design intent	A specified behavior that has not been established as live, measured, or production-verified	Evidence state

Terminology conflict rule

If a new feature introduces a term that overlaps an existing metric, credential, or product surface, the implementation and document must either reuse the canonical term or explicitly qualify the new term. Similar-sounding outcomes must not be silently merged.

PART 9 · IP Pre-Registration Screening

9.1 Status

No live register search was run for this document. This section is a preparatory candidate inventory only. It does not state that a name is clear, safe, available, protectable, or free of conflict. It does not provide a patentability opinion, legal advice, clearance, filing, or freedom-to-operate conclusion.

9.2 Candidate terms surfaced by the current product

CANDIDATE	SURFACE / CONTEXT	CANDIDATE TYPE	SCREENING STATUS
ARK	Platform and identity name	Brand / mark	UNSEARCHED
ARK Report	Report artifact	Product term	UNSEARCHED
ARK Guide	Report companion	Product term	UNSEARCHED
MERLIN	Completion-assurance guide	Brand / product term	UNSEARCHED
Context Craft	Skill/certification vocabulary	Product term	UNSEARCHED
CCMI	Scoring metric	Coined metric term	UNSEARCHED
JST	Scoring metric	Acronym / metric term	UNSEARCHED
CCGE	Skill-game engine	Acronym / product term	UNSEARCHED
SPHINX	Marketplace engine	Brand / product term	UNSEARCHED
GUIN+	Identity/profile layer	Brand / product term	UNSEARCHED
HIVE	Marketplace quality gate	Product/governance term	UNSEARCHED
MERLIN guidance model	Client-side mechanism	Technical mechanism	PATENT LANDSCAPE NOT RUN

9.3 Required next screening record

If a licensed practitioner or explicitly authorized IP workflow is later engaged, record at minimum:

- register(s) queried;
- jurisdiction and classification scope;
- query date;
- exact candidate and spelling variants;
- result identifiers and source links;
- whether the result is a preliminary search or landscape pass;
- the standing non-advisory caveat.

Do not run filing-draft or legal-correspondence generation as a default part of this living PDD.

PART 10 · Living Delta and Review Rules

10.1 Baseline delta from prior current PDD material

This document supersedes neither historical export. It establishes a newer current-state baseline that must be maintained going forward. The significant current-state additions or confirmations are:

- Current code inventory and 24-flag Stage 1 posture.
- ARK Resume, verification documents, confirmation workflow, matchmaking, workforce filters/search/nudges, HR connection testing, and AI model preference surfaces.
- ARK Report plus the renamed ARK Guide.
- MERLIN persistent copilot and `/merlin` guide with honest, preference-based next actions.
- Migration history through `0023_preferred_ai_model.sql`.
- The current test/typecheck evidence recorded in Part 6.

10.2 Drift bands

BAND	MEANING	RESPONSE
GREEN	Current PDD and code agree on route, schema, and behavior; no material unverified claim added	Routine review at next merged increment
AMBER	A route, flag, schema, or outcome changed and the PDD has not yet been reconciled	Human review required before treating this document as current
RED	PDD claims a removed/unsupported capability, hides a security boundary, misstates a credential, or converts design intent into fact	Treat the affected section as invalid; re-baseline from code and evidence

10.3 Update protocol

1. Recount route, page, migration, flag, and test inventory.

2. Read current `replit.md`, schema, flags, route declarations, and changed feature modules.
3. Update the smallest affected ATLAS parts while preserving the 10-part structure.
4. Mark new external, production, financial, regulatory, and IP claims with an evidence state.
5. Run `npm run check` and `npm test`.
6. Run `npm run pdd:render` when the current markdown is changed and verify the generated PDF.
7. Record the new snapshot date and a concise living delta.

10.4 Present verification record

CHECK	RESULT
TypeScript	`npm run check` passed
Full automated suite	`npm test` passed; 293 tests passed
Development workflow	Running on port 5000 at document assembly
Production deployment	Not asserted
Live external IP search	Not run
Live business metrics	Not supplied

> ***Closing statement:*** *This is an ATLAS 360 living PDD of the ARK codebase as presently observed. It is designed to be updated from the repository, not trusted after the repository moves.*

© 2026 Oluseye Shay Amusa · Koncentric Labs Inc. · ATANDA Publications Forum
Prepared in the ATLAS 360 PLAN structure. No certification or legal conclusion is implied.